1. **Архитектура bare-metal C прошивки наносекундного генератора холодной плазмы (STM32G474 + HRTIM + CMSIS)**

Архитектурные правила структурирования проекта и изоляция критических секций

Принципы модульной декомпозиции проекта

Разработка управляющего микрокода для импульсных высоковольтных плазменных генераторов атмосферного давления требует детерминированного выполнения инструкций с точностью до отдельных тактов процессора. Использование высокоуровневых слоев абстракции аппаратуры (HAL, LL) в цепях непосредственного формирования затворных сигналов недопустимо по причине скрытых накладных расходов, непредсказуемой латентности функций обработки прерываний и избыточных проверок состояний.

Архитектура проекта разделяется на изолированные логические уровни:

- Уровень аппаратно-зависимых драйверов BSP (Board Support Package), оперирующий напрямую регистрами периферии через заголовочные файлы CMSIS Core и Device.
- Уровень алгоритмического ядра (App), содержащий конечный автомат (FSM), расчеты защитных интервалов и обработчики аварий.
- Уровень пользовательского интерфейса и отображения телеметрии (UI), функционирующий в некритичном временном домене.

Модуль / Каталог	**Файлы реализации**	**Назначение и функциональные границы**	**Доступ к регистрам периферии**
Core/	startup_stm32g474xx.s, system_stm32g4xx.c	Векторная таблица, системное тактирование 170 МГц, FPU	FLASH, PWR, RCC
Bsp/Gpio	bsp_gpio.c, bsp_gpio.h	Инициализация распиновки, перевод свободных выводов в Analog	GPIOA, GPIOB, GPIOC

Модуль / Каталог	Файлы реализации	Назначение и функциональные границы	Доступ к регистрам периферии
Bsp/Hrtim	bsp_hrtim.c, bsp_hrtim.h	Настройка DLL 5.44 ГГц, Timer A, B, C, Master, Fault 1, EEV	HRTIM1
Bsp/Analog	bsp_analog.c, bsp_analog.h	Конфигурация COMP1/2/3, DAC1_CH1, ADC1 (дифф. режим)	COMP, DAC1, ADC1, ADC12_COMMON
Bsp/Inputs	bsp_inputs.c, bsp_inputs.h	Фильтрация входов EC11, энкодер TIM3, внешние прерывания	TIM3, EXTI, SYSCFG
App/Fsm	fsm_engine.c, fsm_engine.h	Конечный автомат генератора, синхронный останов, защиты	Нет (через inline-вызовы BSP)
Ui/Display	ssd1306_spi.c, ssd1306_spi.h	Низкоуровневый драйвер SPI2, статический кадровый буфер	SPI2, GPIOB, GPIOC
Ui/Telemetry	ui_telemetry.c, ui_telemetry.h	Отрисовка экранов телеметрии и шагов Step-by-Step меню	Нет (работа с буфером кадра)

Изоляция критических секций и управление латентностью

В импульсно-периодических источниках плазмы наносекундный строб разряжает промежуточный емкостный накопитель через первичную обмотку

автотрансформатора с крутизной нарастания тока $dI/dt > 1$ А/нс и градиентом потенциала $dV/dt > 50$ В/нс. Любая непредвиденная задержка при выключении силовых транзисторов или ложное срабатывание детектора нуля влекут за собой сквозные токи и тепловой пробой полупроводниковых структур. Для гарантии абсолютного временного детерминизма внедряются следующие правила межмодульного взаимодействия:

2. **Отказ от глобального маскирования прерываний** __disable_irq()**:** Полное отключение прерываний ядра Cortex-M4F способно задержать исполнение аварийного обработчика перенапряжения или превышения тока. Для создания атомарных участков применяется динамическое изменение порога приоритета через регистр BASEPRI. Это исключает вмешательство фонового прерывания системного таймера SysTick или дисплея, сохраняя вытесняющую способность за аппаратно-критичными авариями HRTIM.
3. **Атомарная модификация управляющих регистров:** Внутри критических контуров запрещено применение неатомарных операций чтения-модификации-записи (|= и &= ~) над разделяемыми регистрами периферии. Изменение состояний линий ввода-вывода и стробов таймеров выполняется исключительно через выделенные регистры атомарной записи: BSRR для портов GPIO, SETx1R/RSTx1R для выходов HRTIM и ODISR/OENR для мгновенного снятия генерации.
4. **Барьеры памяти (**__DMB()**,** __DSB()**,** __ISB()**):** После конфигурации тактирования, калибровки блока задержки DLL таймера высокого разрешения и переключения источников опорных напряжений обязательно применение инструкций синхронизации данных и потока команд для исключения опережающего исполнения инструкций конвейером Cortex-M4F до фактической записи данных в регистры периферии.
5. **Иерархия приоритетов NVIC:**
    - Приоритет 0 (абсолютный приоритет): HRTIM1_FLT_IRQHandler (аппаратный останов по аварии перенапряжения OVP > 900 В).
    - Приоритет 1: COMP1_2_3_IRQHandler (события отсечки напряжения шины накачки 780 В).
    - Приоритет 2: HRTIM1_Master_IRQHandler (синхронизированное безопасное отключение накачки в нуле тока).
    - Приоритет 4: EXTI3_IRQHandler (силовой курок KEY_FIRE).
    - Приоритет 5: EXTI2_IRQHandler (кнопка энкодера).
    - Приоритет 6: SysTick_Handler (квант автомата FSM 1 мс, опрос АЦП, медленная телеметрия).

Электромагнитная совместимость и подавление наводок

Высоковольтный разряд в сопле генерирует мощные синфазные электромагнитные импульсы, способные наводить паразитные потенциалы на высокоимпедансные цифровые проводники. Все неиспользуемые физические выводы микроконтроллера при начальной загрузке программно настраиваются в аналоговый режим без подтяжек (MODER = 11b, PUPDR = 00b). Это аппаратно отключает входные триггеры Шмитта входных каскадов ввода-вывода, предотвращая возникновение сквозных паразитных токов через защитные диоды кристалла и ликвидируя риск паразитного логического переключения ядра от емкостных наводок.

Аппаратные конфигурации CMSIS для HRTIM1 и аналоговой периферии

Распределение выводов WeAct Studio STM32G474CEU6

В таблице представлена полная привязка аппаратных ресурсов микроконтроллера с параметрами быстродействия выходных каскадов и типами аналоговых цепей.

Физический вывод	**Порт GPIO**	**Альтернативная функция / Режим**	**Периферийный блок**	**Назначение в схеме генератора**
Pin 10	PA8	AF13, Very High Speed, Push-Pull, No-Pull	HRTIM1_CHC1	Строб 14.0 нс на затворный драйвер SiC UCC27511
Pin 13	PA11	AF13, Very High Speed, Push-Pull, No-Pull	HRTIM1_CHA1	Затвор транзистора M1 ZVS (CSD18534Q5A)
Pin 12	PA10	AF13, Very High Speed, Push-Pull, No-Pull	HRTIM1_CHB1	Затвор транзистора M2 ZVS (CSD18534Q5A)
Pin 41	PA1	Analog, No-Pull	COMP1_INP	Детектор нуля стока M1 (EEV1 сброс Timer A)

Физический вывод	Порт GPIO	Альтернативная функция / Режим	Периферийный блок	Назначение в схеме генератора
Pin 35	PA7	Analog, No-Pull	COMP2_INP	Детектор нуля стока M2 (EEV2 сброс Timer B)
Pin 42	PA2	Analog, No-Pull	COMP3_INP / ADC1_IN3	Неинвертирующий замер HV шины (AMC1311 VOUTP)
Pin 39	PA3	Analog, No-Pull	ADC1_IN4	Инвертирующий опорный вход AMC1311 VOUTN
Pin 44	PA0	Analog, No-Pull	ADC1_IN1	Контроль батареи 2S LiPo (делитель 100k/47.5k)
Pin 33	PB0	Output, Low Speed, Push-Pull, Pull-Down	GPIO	Ключ соленоида газового клапана AO3400A
Pin 30	PB13	AF5, High Speed, Push-Pull, No-Pull	SPI2_SCK	Тактирование OLED-контроллера SSD1306
Pin 32	PB15	AF5, High Speed, Push-Pull, No-Pull	SPI2_MOSI	Линия передачи данных OLED SSD1306
Pin 29	PB12	Output, High Speed, Push-Pull, No-Pull	GPIO	Выбор ведомого (CS) дисплея SSD1306

Физический вывод	Порт GPIO	Альтернативная функция / Режим	Периферийный блок	Назначение в схеме генератора
Pin 31	PB14	Output, High Speed, Push-Pull, No-Pull	GPIO	Линия команда/данные (DC) дисплея SSD1306
Pin 34	PC6	Output, High Speed, Push-Pull, No-Pull	GPIO	Аппаратный сброс (RST) дисплея SSD1306
Pin 19	PB4	AF2, Input, Pull-Up, фильтр IC1F=0xF	TIM3_CH1	Квадратурный канал A энкодера EC11
Pin 20	PB5	AF2, Input, Pull-Up, фильтр IC2F=0xF	TIM3_CH2	Квадратурный канал B энкодера EC11
Pin 31	PB2	Input, Pull-Up, EXTI2 Falling/Rising	EXTI2	Тактовая кнопка энкодера EC11
Pin 18	PB3	Input, Pull-Up, EXTI3 Falling/Rising	EXTI3	Кнопка курка запуска плазмы KEY_FIRE

CMSIS-инициализация портов GPIO и аналоговых трактов

Модуль ввода-вывода конфигурирует системные шины и переводит свободные линии кристалла в аналоговое состояние для максимального снижения чувствительности к импульсным электромагнитным помехам.

C

#**include** "stm32g4xx.h"

#**include** <stdint.h>

#**include** <stdbool.h>

```
void bsp_gpio_init(void)
{
  // Включение тактирования портов A, B, C
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN | RCC_AHB2ENR_GPIOBEN | RCC_AHB2ENR_GPIOCEN;
  __DSB();

  // 1. Перевод всех выводов по умолчанию в Analog No-Pull для снижения энергопотребления и наводок
  // В чипах STM32G4 большинство пинов стартуют в Analog, гарантируем отсутствие подтяжек
  GPIOA->PUPDR = 0x00000000;
  GPIOB->PUPDR = 0x00000000;
  GPIOC->PUPDR = 0x00000000;

  // 2. Силовые импульсные выходы HRTIM: PA8 (CHC1), PA10 (CHB1), PA11 (CHA1)
  // Альтернативная функция AF13, предельное быстродействие Very High Speed (11b), No-Pull
  GPIOA->MODER &= ~((3U << (8 * 2)) | (3U << (10 * 2)) | (3U << (11 * 2)));
  GPIOA->MODER |=  ((2U << (8 * 2)) | (2U << (10 * 2)) | (2U << (11 * 2)));
  GPIOA->AFR[1] |= (13U << ((8 - 8) * 4)) | (13U << ((10 - 8) * 4)) | (13U << ((11 - 8) * 4));
  GPIOA->OSPEEDR |= ((3U << (8 * 2)) | (3U << (10 * 2)) | (3U << (11 * 2)));

  // 3. Аналоговые измерительные входы: PA0 (ADC1_IN1), PA1 (COMP1_INP),
  // PA2 (COMP3_INP/ADC1_IN3), PA3 (ADC1_IN4), PA7 (COMP2_INP)
  GPIOA->MODER |= ((3U << (0 * 2)) | (3U << (1 * 2)) | (3U << (2 * 2)) |
                   (3U << (3 * 2)) | (3U << (7 * 2)));

  // 4. Газовый соленоид: PB0 -> General Purpose Output, Low Speed, Pull-Down
```

```
  GPIOB->MODER &= ~(3U << (0 * 2));
  GPIOB->MODER |=  (1U << (0 * 2));
  GPIOB->OSPEEDR &= ~(3U << (0 * 2));
  GPIOB->PUPDR &= ~(3U << (0 * 2));
  GPIOB->PUPDR |=  (2U << (0 * 2)); // Аппаратная стяжка к земле
  GPIOB->BSRR = GPIO_BSRR_BR0;      // Гарантированное закрытие клапана

  // 5. Дисплей SSD1306 SPI2: PB13 (SCK, AF5), PB15 (MOSI, AF5), PB12 (CS, Out), PB14 (DC, Out), PC6 (RST, Out)
  GPIOB->MODER &= ~((3U << (12 * 2)) | (3U << (13 * 2)) | (3U << (14 * 2)) | (3U << (15 * 2)));
  GPIOB->MODER |=  ((1U << (12 * 2)) | (2U << (13 * 2)) | (1U << (14 * 2)) | (2U << (15 * 2)));
  GPIOB->AFR[1] |= ((5U << ((13 - 8) * 4)) | (5U << ((15 - 8) * 4)));
  GPIOB->OSPEEDR |= ((3U << (12 * 2)) | (3U << (13 * 2)) | (3U << (14 * 2)) | (3U << (15 * 2)));
  GPIOB->BSRR = GPIO_BSRR_BS12; // CS High (шина свободна)

  GPIOC->MODER &= ~(3U << (6 * 2));
  GPIOC->MODER |=  (1U << (6 * 2));
  GPIOC->OSPEEDR |= (3U << (6 * 2));
  GPIOC->BSRR = GPIO_BSRR_BS6;  // RST High

  // 6. Квадратурный энкодер: PB4 (TIM3_CH1, AF2), PB5 (TIM3_CH2, AF2) с аппаратной подтяжкой
  GPIOB->MODER &= ~((3U << (4 * 2)) | (3U << (5 * 2)));
  GPIOB->MODER |=  ((2U << (4 * 2)) | (2U << (5 * 2)));
  GPIOB->AFR[0] |= ((2U << (4 * 4)) | (2U << (5 * 4)));
  GPIOB->PUPDR &= ~((3U << (4 * 2)) | (3U << (5 * 2)));
  GPIOB->PUPDR |=  ((1U << (4 * 2)) | (1U << (5 * 2)));

  // 7. Кнопки: PB2 (кнопка энкодера), PB3 (курок KEY_FIRE) с внутренней подтяжкой к питанию
```

```
  GPIOB->MODER &= ~((3U << (2 * 2)) | (3U << (3 * 2)));
  GPIOB->PUPDR &= ~((3U << (2 * 2)) | (3U << (3 * 2)));
  GPIOB->PUPDR |=  ((1U << (2 * 2)) | (1U << (3 * 2)));
}
C
void bsp_analog_init(void)
{
  // Активация тактирования аналоговых интерфейсов
  RCC->APB1ENR1 |= RCC_APB1ENR1_DAC1EN;
  RCC->APB2ENR  |= RCC_APB2ENR_SYSCFGEN;
  RCC->AHB2ENR  |= RCC_AHB2ENR_ADC12EN;
  __DSB();

  // ---------------------------------------------------------------------------
  // 1. Внутренний ЦАП DAC1_CH1: Опорное напряжение отсечки COMP3
  // Выход буферизован и направлен на внутреннюю матрицу компараторов
  // ---------------------------------------------------------------------------
  DAC1->CR &= ~DAC_CR_EN1;
  DAC1->MCR &= ~DAC_MCR_MODE1_Msk; // Режим Normal: подключение к внутренней периферии
  DAC1->DHR12R1 = 2710U;         // Уставка 780 В на шине (2.184 В на входе PA2)
  DAC1->CR |= DAC_CR_EN1;
  for (volatile uint32_t i = 0; i < 2000; i++) { __NOP(); } // Ожидание стабилизации

  // ---------------------------------------------------------------------------
  // 2. Компараторы перехода стоков через ноль COMP1 и COMP2
  // Инвертирующие входы подключены к 1/4 VREFINT (~0.303 В)
  // ---------------------------------------------------------------------------
  // COMP1: Неинвертирующий вход PA1 (INP0), гистерезис 10 мВ
```

```
  COMP1->CSR = (0U << COMP_CSR_INPSEL_Pos) |
          (4U << COMP_CSR_INMSEL_Pos) |
          (1U << COMP_CSR_HYST_Pos)   |
          COMP_CSR_EN;

  // COMP2: Неинвертирующий вход PA7 (INP1), гистерезис 10 мВ
  COMP2->CSR = (1U << COMP_CSR_INPSEL_Pos) |
          (4U << COMP_CSR_INMSEL_Pos) |
          (1U << COMP_CSR_HYST_Pos)   |
          COMP_CSR_EN;

  // ---------------------------------------------------------------------
  // 3. Компаратор отсечки шины COMP3
  // Неинвертирующий вход PA2 (INP0), инвертирующий вход DAC1_CH1
  // ---------------------------------------------------------------------
  COMP3->CSR = (0U << COMP_CSR_INPSEL_Pos) |
          (7U << COMP_CSR_INMSEL_Pos) |
          (2U << COMP_CSR_HYST_Pos)   | // Гистерезис 20 мВ
          COMP_CSR_EN;

  // ---------------------------------------------------------------------
  // 4. Дифференциальный АЦП ADC1 (AMC1311 VOUTP/VOUTN) и мониторинг батареи
  // ---------------------------------------------------------------------
  ADC1->CR &= ~ADC_CR_DEEPPWD;
  ADC1->CR |= ADC_CR_ADVREGEN;
  for (volatile uint32_t i = 0; i < 4000; i++) { __NOP(); } // Прогрев внутреннего LDO

  ADC12_COMMON->CCR = (1U << ADC_CCR_CKMODE_Pos); // Синхронное тактирование fHCLK/1
```

```
  // Запуск калибровки дифференциального тракта
  ADC1->CR |= ADC_CR_ADCALDIF;
  ADC1->CR |= ADC_CR_ADCAL;
  while (ADC1->CR & ADC_CR_ADCAL);

  // Выбор дифференциального режима для Канала 3 (PA2) в паре с Каналом 4 (PA3)
  ADC1->DIFSEL |= ADC_DIFSEL_DIFSEL_3;

  // Включение блока преобразователя
  ADC1->ISR |= ADC_ISR_ADRDY;
  ADC1->CR |= ADC_CR_ADEN;
  while (!(ADC1->ISR & ADC_ISR_ADRDY));

  // Настройка регулярной последовательности: Ранг 1 = Канал 3 (AMC1311), Ранг 2 = Канал 1 (PA0, LiPo)
  ADC1->SQR1 = (1U << ADC_SQR1_L_Pos) | (3U << ADC_SQR1_SQ1_Pos) | (1U << ADC_SQR1_SQ2_Pos);

  // Время интегрирования заряда: 47.5 тактов на канал
  ADC1->SMPR1 = (4U << ADC_SMPR1_SMP1_Pos) | (4U << ADC_SMPR1_SMP3_Pos);

  // Аппаратный аналоговый компаратор AWD1: дублирующая защита от OVP (> 900 В, код 2920)
  ADC1->TR1 = (2920U << ADC_TR1_HT1_Pos) | (0U << ADC_TR1_LT1_Pos);
  ADC1->CFGR |= ADC_CFGR_AWD1EN | ADC_CFGR_AWD1SGL | (3U << ADC_CFGR_AWD1CH_Pos);
}
C
void bsp_inputs_init(void)
```

```
{
  RCC->APB1ENR1 |= RCC_APB1ENR1_TIM3EN;
  __DSB();

  // 1. Аппаратный квадратурный декодер TIM3 (режим Encoder Mode 3)
  // Подсчет импульсов по фронтам каналов TI1 и TI2 с максимальной цифровой фильтрацией ICxF = 0xF
  TIM3->SMCR = (3U << TIM_SMCR_SMS_Pos);
  TIM3->CCMR1 = (1U << TIM_CCMR1_CC1S_Pos) | (0xFU << TIM_CCMR1_IC1F_Pos) |
          (1U << TIM_CCMR1_CC2S_Pos) | (0xFU << TIM_CCMR1_IC2F_Pos);
  TIM3->CCER = 0x0000; // Прямая полярность счета
  TIM3->ARR = 0xFFFF;
  TIM3->CNT = 0x8000;  // Смещение в середину диапазона 16-битного регистра
  TIM3->CR1 = TIM_CR1_CEN;

  // 2. Внешние прерывания EXTI2 (кнопка энкодера) и EXTI3 (курок KEY_FIRE)
  SYSCFG->EXTICR[0] &= ~(SYSCFG_EXTICR1_EXTI2_Msk | SYSCFG_EXTICR1_EXTI3_Msk);
  SYSCFG->EXTICR[0] |=  (SYSCFG_EXTICR1_EXTI2_PB | SYSCFG_EXTICR1_EXTI3_PB);

  // EXTI2: Срабатывание по спаду импульса (нажатие кнопки)
  EXTI->IMR1  |= EXTI_IMR1_IM2;
  EXTI->FTSR1 |= EXTI_FTSR1_FT2;

  // EXTI3: Отслеживание нажатия (спад) и отпускания (фронт) для замера длительности удержания
  EXTI->IMR1  |= EXTI_IMR1_IM3;
  EXTI->FTSR1 |= EXTI_FTSR1_FT3;
  EXTI->RTSR1 |= EXTI_RTSR1_TR3;

  NVIC_SetPriority(EXTI2_IRQn, 5);
```

```
  NVIC_EnableIRQ(EXTI2_IRQn);
  NVIC_SetPriority(EXTI3_IRQn, 4);
  NVIC_EnableIRQ(EXTI3_IRQn);
}
```

Настройка HRTIM1: генерация 345 кГц ZVS, наносекундный строб SiC и аварийная отсечка

Таймер высокого разрешения HRTIM1 представляет собой ядро системы генерации. При тактовой частоте шины 170 МГц встроенная схема с автоподстройкой задержки DLL умножает базовую частоту на 32, достигая эффективного разрешения 5.44 ГГц с шагом дискретизации квантов времени:

$$t_{\text{step}} = \frac{1}{170\ \text{МГц} \times 32} \approx 183.82\ \text{пс}$$

Математический расчет базовых временных интервалов таймера:

- Частота резонансной накачки ZVS: $f_{\text{res}} = 345$ кГц.
- Период колебаний: $T_{\text{res}} = \frac{1}{345\ \text{кГц}} \approx 2.89855$ мкс.
- Период в дискретах HRTIM: $N_{\text{period}} = \frac{2.89855\ \text{мкс}}{183.82\ \text{пс}} = 15768$ тиков.
- Полупериод резонанса: $N_{\text{half}} = \frac{15768}{2} = 7884$ тика.
- Длительность аппаратного перекрытия фаз (Overlap 50 нс) для мягкого замыкания дросселя L_6: $N_{\text{overlap}} = \frac{50\ \text{нс}}{183.82\ \text{пс}} = 272$ тика.
- Полное время открытого состояния ключей ZVS: $7884 + 272 = 8156$ тиков.
- Длительность управляющего импульса SiC: строго 14.0 нс $\rightarrow N_{\text{sic}} = \frac{14.0\ \text{нс}}{183.82\ \text{пс}} \approx 76.16 \rightarrow 76$ тиков.

C

```
void bsp_hrtim_init(void)
{
  RCC->APB2ENR |= RCC_APB2ENR_HRTIM1EN;
  __DSB();

  // 1. Запуск и стабилизация блока DLL (5.44 ГГц)
  HRTIM1->sCommonRegs.DLLCR = HRTIM_DLLCR_CAL;
```

```
  while (!(HRTIM1->sCommonRegs.ISR & HRTIM_ISR_DLLRDY));
  // Непрерывная фоновая калибровка для компенсации термического дрейфа кристаллической решетки
  HRTIM1->sCommonRegs.DLLCR = HRTIM_DLLCR_CALEN | (3U << HRTIM_DLLCR_CALRTE_Pos);

  // 2. Внешние события EEV: привязка компараторов к аппаратным событиям таймера
  // EEV1: COMP1 (сброс Timer A по спаду стока M1), Fast Mode, восходящий фронт
  // EEV2: COMP2 (сброс Timer B по спаду стока M2), Fast Mode, восходящий фронт
  HRTIM1->sCommonRegs.EECR1 =
    (1U << HRTIM_EECR1_EE1SRC_Pos) | HRTIM_EECR1_EE1FAST | (1U << HRTIM_EECR1_EE1SNS_Pos) |
    (1U << HRTIM_EECR1_EE2SRC_Pos) | HRTIM_EECR1_EE2FAST | (1U << HRTIM_EECR1_EE2SNS_Pos);

  // EEV3: COMP3 (достижение 780 В на шине HV), уровень высокий
  HRTIM1->sCommonRegs.EECR2 =
    (1U << HRTIM_EECR2_EE3SRC_Pos) | (0U << HRTIM_EECR2_EE3SNS_Pos);

  // 3. Аппаратная защита Fault 1: аварийное отключение выходов по перенапряжению OVP (> 900 В)
  // Аппаратный тракт гасит выходы менее чем за 25 нс в обход ядра
  HRTIM1->sCommonRegs.FLTINR1 = HRTIM_FLTINR1_FLT1E | HRTIM_FLTINR1_FLT1SRC;

  // 4. Master Timer: Базовая синхронизирующая шкала 345 кГц
  HRTIM1->sMasterRegs.MCR = HRTIM_MCR_CONT | HRTIM_MCR_PREEN;
  HRTIM1->sMasterRegs.MPER = 15768U;
  HRTIM1->sMasterRegs.MCMP1R = 7884U; // Фазовый сдвиг 180 градусов

  // Разрешение прерываний для координации синхронного останова накачки
```

```
  HRTIM1->sMasterRegs.MIER = HRTIM_MIER_MCMP1IE | HRTIM_MIER_MPERIE;

  // 5. Timer A: Ключ M1 ZVS (PA11, CHA1)
  HRTIM1->sTimerxRegs[0].TIMxCR = HRTIM_TIMCR_PREEN | HRTIM_TIMCR_CONT;
  HRTIM1->sTimerxRegs[0].PERxR  = 15768U;
  HRTIM1->sTimerxRegs[0].CMP1xR = 8156U; // Длительность импульса с учетом оверлапа 50 нс

  HRTIM1->sTimerxRegs[0].SETx1R = HRTIM_SET1R_MSTPER; // Включение по началу периода Master
  HRTIM1->sTimerxRegs[0].RSTx1R = HRTIM_RST1R_CMP1;   // Выключение по достижению порога

  // Адаптивная синхронизация: коррекция периода при реальном обнаружении перехода через ноль
  HRTIM1->sTimerxRegs[0].RSTAR  = HRTIM_RSTAR_EEV1;

  HRTIM1->sTimerxRegs[0].OUTxR  = (0U << HRTIM_OUTR_FAULT1_Pos); // В неактивный уровень при аварии
  HRTIM1->sTimerxRegs[0].FLTxR  = HRTIM_FLTR_FLT1EN;

  // 6. Timer B: Ключ M2 ZVS (PA10, CHB1)
  HRTIM1->sTimerxRegs[1].TIMxCR = HRTIM_TIMCR_PREEN | HRTIM_TIMCR_CONT;
  HRTIM1->sTimerxRegs[1].PERxR  = 15768U;
  HRTIM1->sTimerxRegs[1].CMP1xR = 7884U; // Момент включения (сдвиг 180 градусов)
  HRTIM1->sTimerxRegs[1].CMP2xR = 272U;  // Выключение со смещением перекрытия

  HRTIM1->sTimerxRegs[1].SETx1R = HRTIM_SET1R_MSTCMP1;
  HRTIM1->sTimerxRegs[1].RSTx1R = HRTIM_RST1R_CMP2;

  HRTIM1->sTimerxRegs[1].RSTAR  = HRTIM_RSTAR_EEV2;
```

```
  HRTIM1->sTimerxRegs[1].OUTxR  = (0U << HRTIM_OUTR_FAULT1_Pos);
  HRTIM1->sTimerxRegs[1].FLTxR  = HRTIM_FLTR_FLT1EN;

  // 7. Timer C: Формирователь одиночного наносекундного строба SiC (PA8, CHC1)
  // Режим One-Shot non-retriggerable: строго 14.0 нс (76 квантов DLL)
  HRTIM1->sTimerxRegs[2].TIMxCR = HRTIM_TIMCR_PREEN; // CONT = 0 (одиночный импульс)
  HRTIM1->sTimerxRegs[2].PERxR  = 1000U;
  HRTIM1->sTimerxRegs[2].CMP1xR = 76U; // Длительность проводящего состояния ключа

  HRTIM1->sTimerxRegs[2].SETx1R = HRTIM_SET1R_SST;   // Программный запуск строба
  HRTIM1->sTimerxRegs[2].RSTx1R = HRTIM_RST1R_CMP1;  // Аппаратное запирание

  HRTIM1->sTimerxRegs[2].OUTxR  = (0U << HRTIM_OUTR_FAULT1_Pos);
  HRTIM1->sTimerxRegs[2].FLTxR  = HRTIM_FLTR_FLT1EN;

  // Активация физических выходов в контроллере выходов
  HRTIM1->sCommonRegs.OENR = HRTIM_OENR_TA1OEN | HRTIM_OENR_TB1OEN | HRTIM_OENR_TC1OEN;

  NVIC_SetPriority(HRTIM1_FLT_IRQn, 0);
  NVIC_EnableIRQ(HRTIM1_FLT_IRQn);
  NVIC_SetPriority(HRTIM1_Master_IRQn, 2);
  NVIC_EnableIRQ(HRTIM1_Master_IRQn);
}
```

Механизм синхронного останова ZVS накачки

Коммутация токового резонансного пуш-пула (Current-Fed Push-Pull) на базе входного дросселя $L_6 = 22$ мкГн сопряжена с риском возникновения высоковольтного индуктивного выброса. Если прервать протекание тока в первичной полуобмотке в

произвольный момент полупериода, когда в сердечнике дросселя запасена максимальная энергия $\frac{1}{2}LI^2$, накопленный заряд вызовет лавинный пробой защитных супрессоров 1.5SMC39A и их термическую деградацию.

Для исключения перенапряжения отключение выходов CHA1 и CHB1 синхронизируется с естественным окончанием резонансного такта:

C

```
static volatile bool zvs_stop_requested = false;
static volatile bool zvs_active_flag    = false;


void COMP1_2_3_IRQHandler(void)
{
  // Компаратор COMP3 зафиксировал превышение порога 780 В на шине HV
  if (COMP3->CSR & COMP_CSR_VALUE)
  {
    zvs_stop_requested = true;
  }
}


void HRTIM1_Master_IRQHandler(void)
{
  uint32_t misr = HRTIM1->sMasterRegs.MISR;


  // Проверка наступления границы полупериода (момент естественного перехода через ноль)
  if ((misr & HRTIM_MISR_MCMP1) || (misr & HRTIM_MISR_MPER))
  {
    HRTIM1->sMasterRegs.MICR = misr; // Сброс прерывания


    if (zvs_stop_requested && zvs_active_flag)
    {
```

```
        // Мгновенное снятие затворных импульсов строго в фазе нулевого тока
        HRTIM1->sCommonRegs.ODISR = HRTIM_ODISR_TA1ODIS | HRTIM_ODISR_TB1ODIS;
        // Останов тактовых генераторов плеч накачки
        HRTIM1->sMasterRegs.MCR &= ~(HRTIM_MCR_MCEN | HRTIM_MCR_TACEN | HRTIM_MCR_TBCEN);

        zvs_active_flag   = false;
        zvs_stop_requested = false;
      }
    }
}

void HRTIM1_FLT_IRQHandler(void)
{
  // Аппаратная катастрофическая авария OVP (> 900 В)
  HRTIM1->sCommonRegs.ICR = HRTIM_ICR_FLT1C;
  HRTIM1->sCommonRegs.ODISR = HRTIM_ODISR_TA1ODIS | HRTIM_ODISR_TB1ODIS | HRTIM_ODISR_TC1ODIS;
  GPIOB->BSRR = GPIO_BSRR_BR0; // Немедленная отсечка подачи газа
}
```

Архитектура конечного автомата (FSM) и алгоритмы защит

Состояния автомата и временные нормативы

Логика управления генератором строится вокруг восьми состояний, обеспечивающих безопасность плазмохимического реактора, защиту силовых полупроводников от перегрева и исключение пробоя изоляции кварцевого канала.

Состояние FSM	**Входное условие**	**Активные исполнительные узлы**	**Выходное условие перехода**
IDLE	Исходное состояние или сброс	Все каскады обесточены, клапан закрыт	Нажатие курка KEY_FIRE
PREFLOW	Переход из IDLE	Открыт газовый клапан PB0 (соленоид 0520D)	Истечение интервала предпродувки 150 мс
CHARGE	Завершение PREFLOW или повтор	Включена ZVS накачка 345 кГц, активен замер ADC1	Достижение 780 В (COMP3) или таймаут 250 мкс
FIRING	Синхронный останов ZVS	Строб SiC 14.0 нс, автотрансформатор T2	Завершение импульса 14 нс, оценка кнопки курка
POSTFLOW	Отпускание курка / стоп	Клапан PB0 удерживается открытым для продувки	150 мс (Single), 300 мс (Hold), 500 мс (Limit)
THERMAL_LOCK	Превышение лимита 15.0 с	Все каскады заблокированы, сопло охлаждается	Истечение таймаута принудительного лока 5.0 с
FAULT	Авария OVP, UVLO или таймаут	Генерация остановлена, клапан закрыт	Длительное удержание (> 1.5 с) кнопки энкодера

Состояние FSM	Входное условие	Активные исполнительные узлы	Выходное условие перехода
STEP_DEBUG	Выбор в инженерном меню	Конфигурация в зависимости от шага STEP 0..6	Команда навигации через энкодер EC11

Переходы между состояниями детерминированы физическими ограничениями системы. После перехода из состояния ожидания IDLE соленоид открывает подачу аргона на 150 мс (PREFLOW), вытесняя атмосферный кислород из кварцевого зазора сопла, что необходимо для создания стабильного низкотемпературного стримерного разряда без риска термической контракции в дугу.

По завершении продувки инициируется фаза накачки CHARGE. При достижении расчетного потенциала 780 В компаратор COMP3 выдает запрос на синхронное отключение, таймер Master завершает текущий полупериод, и формируется технологическая пауза 1.5 мкс для рассеяния переходных процессов в сердечнике T1.

Следом в фазе FIRING генерируется одиночный строб длительностью 14.0 нс. Если курок удерживается дольше 350 мс, система переходит в циклический режим генерации пачек импульсов с частотой до 10 кГц.

Встроенный сторожевой таймер непрерывной генерации жестко ограничивает непрерывную работу интервалом 15.0 секунд, предотвращая термический пробой ферритового кольца автотрансформатора T2 и перегрев балластных резисторов клампера C4D02120E. При превышении этого лимита автомат принудительно переходит в фазу продувки 500 мс и блокирует повторный пуск на 5 секунд в состоянии THERMAL_LOCK.

Полный программный модуль конечного автомата

C

```
typedef enum {
  FSM_STATE_IDLE = 0,
  FSM_STATE_PREFLOW,
  FSM_STATE_CHARGE,
  FSM_STATE_FIRING,
  FSM_STATE_POSTFLOW,
  FSM_STATE_THERMAL_LOCK,
```

```
    FSM_STATE_FAULT,
    FSM_STATE_STEP_DEBUG
} fsm_state_t;

typedef enum {
    FAULT_NONE = 0,
    FAULT_OVP_900V,
    FAULT_UVLO_BATTERY,
    FAULT_CHARGE_TIMEOUT
} fsm_fault_t;

typedef struct {
    fsm_state_t state;
    fsm_fault_t fault;
    uint32_t state_timer_ms;
    uint32_t charge_elapsed_us;
    uint32_t continuous_firing_ms;
    uint32_t postflow_limit_ms;
    uint32_t shot_counter;
    uint16_t v_target_bus;
    uint16_t v_measured_bus;
    uint16_t v_battery_mv;
    uint16_t pulse_rep_rate_hz;
    bool trigger_active;
    uint32_t trigger_hold_duration_ms;
    uint8_t current_debug_step;
} plasma_machine_t;

static plasma_machine_t fsm = {
```

```
    .state = FSM_STATE_IDLE,
    .fault = FAULT_NONE,
    .v_target_bus = 780,
    .pulse_rep_rate_hz = 10000,
    .current_debug_step = 0
};

static inline void zvs_activate_power(void)
{
    zvs_stop_requested = false;
    zvs_active_flag    = true;
    HRTIM1->sCommonRegs.OENR |= (HRTIM_OENR_TA1OEN | HRTIM_OENR_TB1OEN);
    HRTIM1->sMasterRegs.MCNTR = 0;
    HRTIM1->sTimerxRegs[0].CNTxR = 0;
    HRTIM1->sTimerxRegs[1].CNTxR = 0;
    HRTIM1->sMasterRegs.MCR |= (HRTIM_MCR_MCEN | HRTIM_MCR_TACEN |
HRTIM_MCR_TBCEN);
}

static inline void sic_strobe_trigger(void)
{
    HRTIM1->sTimerxRegs[2].CNTxR = 0;
    HRTIM1->sTimerxRegs[2].SETx1R |= HRTIM_SET1R_SST;
    fsm.shot_counter++;
}

void fsm_process_tick_1ms(void)
{
    // Защита от глубокого разряда LiPo аккумулятора (< 6.6 В)
```

```
  if (fsm.v_battery_mv < 6600 && fsm.state != FSM_STATE_FAULT)
  {
    fsm.state = FSM_STATE_FAULT;
    fsm.fault = FAULT_UVLO_BATTERY;
    GPIOB->BSRR = GPIO_BSRR_BR0; // Закрыть соленоид
    HRTIM1->sCommonRegs.ODISR = HRTIM_ODISR_TA1ODIS | HRTIM_ODISR_TB1ODIS |
HRTIM_ODISR_TC1ODIS;
    return;
  }

  if (fsm.trigger_active)
  {
    fsm.trigger_hold_duration_ms++;
  }

  switch (fsm.state)
  {
    case FSM_STATE_IDLE:
      GPIOB->BSRR = GPIO_BSRR_BR0;
      fsm.continuous_firing_ms = 0;
      if (fsm.trigger_active)
      {
        GPIOB->BSRR = GPIO_BSRR_BS0; // Открыть предпродувку газом
        fsm.state_timer_ms = 0;
        fsm.state = FSM_STATE_PREFLOW;
      }
      break;

    case FSM_STATE_PREFLOW:
```

```
        fsm.state_timer_ms++;
        if (fsm.state_timer_ms >= 150)
        {
            fsm.state_timer_ms = 0;
            fsm.charge_elapsed_us = 0;
            zvs_activate_power();
            fsm.state = FSM_STATE_CHARGE;
        }
        break;

    case FSM_STATE_CHARGE:
        fsm.charge_elapsed_us += 1000;
        if (!zvs_active_flag) // Накачка синхронно отключена по COMP3
        {
            // Задержка 1.5 мкс для затухания колебаний
            for (volatile int i = 0; i < 40; i++) { __NOP(); }
            sic_strobe_trigger();
            fsm.state = FSM_STATE_FIRING;
        }
        else if (fsm.charge_elapsed_us > 250000) // Таймаут заряда более 250 мкс
        {
            fsm.state = FSM_STATE_FAULT;
            fsm.fault = FAULT_CHARGE_TIMEOUT;
            HRTIM1->sCommonRegs.ODISR = HRTIM_ODISR_TA1ODIS |
HRTIM_ODISR_TB1ODIS;
        }
        break;

    case FSM_STATE_FIRING:
```

```
if (fsm.trigger_active)
{
  fsm.continuous_firing_ms++;
  // Аппаратное ограничение непрерывной работы 15.0 с
  if (fsm.continuous_firing_ms >= 15000)
  {
    fsm.postflow_limit_ms = 500;
    fsm.state_timer_ms = 0;
    fsm.state = FSM_STATE_POSTFLOW;
    break;
  }

  if (fsm.trigger_hold_duration_ms < 350)
  {
    // Режим Single-Shot: переход на постпродувку
    fsm.postflow_limit_ms = 150;
    fsm.state_timer_ms = 0;
    fsm.state = FSM_STATE_POSTFLOW;
  }
  else
  {
    // Режим удержания: повторный цикл накачки и импульса
    fsm.state_timer_ms = 0;
    fsm.charge_elapsed_us = 0;
    zvs_activate_power();
    fsm.state = FSM_STATE_CHARGE;
  }
}
else
```

```
    {
      fsm.postflow_limit_ms = 300;
      fsm.state_timer_ms = 0;
      fsm.state = FSM_STATE_POSTFLOW;
    }
    break;

  case FSM_STATE_POSTFLOW:
    fsm.state_timer_ms++;
    if (fsm.state_timer_ms >= fsm.postflow_limit_ms)
    {
      GPIOB->BSRR = GPIO_BSRR_BR0; // Завершить продувку сопла
      if (fsm.continuous_firing_ms >= 15000)
      {
        fsm.state_timer_ms = 0;
        fsm.state = FSM_STATE_THERMAL_LOCK;
      }
      else
      {
        fsm.state = FSM_STATE_IDLE;
      }
    }
    break;

  case FSM_STATE_THERMAL_LOCK:
    fsm.state_timer_ms++;
    if (fsm.state_timer_ms >= 5000) // Защитная пауза 5.0 с
    {
      fsm.state = FSM_STATE_IDLE;
```

```
        }
        break;

    case FSM_STATE_FAULT:
        HRTIM1->sCommonRegs.ODISR = HRTIM_ODISR_TA1ODIS | HRTIM_ODISR_TB1ODIS
| HRTIM_ODISR_TC1ODIS;
        GPIOB->BSRR = GPIO_BSRR_BR0;
        break;

    case FSM_STATE_STEP_DEBUG:
        break;
  }
}
```

C

```
void EXTI3_IRQHandler(void)
{
  EXTI->PR1 = EXTI_PR1_PIF3;

  if (!(GPIOB->IDR & GPIO_IDR_ID3)) // Спад: курок нажат
  {
    fsm.trigger_active = true;
    fsm.trigger_hold_duration_ms = 0;
  }
  else // Фронт: курок отпущен
  {
    fsm.trigger_active = false;
  }
}
```

```
void EXTI2_IRQHandler(void)
{
  EXTI->PR1 = EXTI_PR1_PIF2;

  static uint32_t press_timestamp = 0;
  if (!(GPIOB->IDR & GPIO_IDR_ID2))
  {
    press_timestamp = fsm.state_timer_ms;
  }
  else
  {
    uint32_t duration = fsm.state_timer_ms - press_timestamp;
    if (duration > 1500) // Длинное удержание (> 1.5 с) для сброса аварий
    {
      if (fsm.state == FSM_STATE_FAULT)
      {
        fsm.fault = FAULT_NONE;
        fsm.state = FSM_STATE_IDLE;
      }
    }
  }
}
```

Пошаговый протокол отладки платы (Step-by-Step Bring-Up)

Процедура первого запуска требует раздельной проверки измерительных цепей, изоляции драйверов и резонансных контуров. Ошибки в фазировке трансформатора T1 или деградация затворных фронтов SiC-транзистора приводят к сквозному пробою дорогостоящих компонентов. Инженерное меню позволяет производить ступенчатую верификацию каждого физического узла.

Этап ввода	Наименование	Состояние силовой части	Проверяемые точки и измерительные приборы	Критерий перехода к следующему шагу
STEP 0	Cold Board Verification	Питание силовой ступени снято	Мультиметр: шина +3.3 В (MP2315), +15.0 В (MT3608), +5.0 В ISO на VDD1 AMC1311	Отклонение всех напряжений вторичных источников в пределах $\pm 2\%$
STEP 1	Valve Integrity Test	Сила обесточена, клапан подключен	Осциллограф Siglent SDS 1202CNL: щуп Siglent PP215 1:1 на затворе ключа AO3400A (PB0)	Напряжение 3.3 В, щелчок соленоида 0520D, отсутствие помех по шине питания
STEP 2	Dry Run Gates Check	Силовая шина 0 В, драйверы включены	Щуп Siglent PP215 10:1 с пружинным заземлением на затворах M1, M2 и SiC	Частота 345 кГц, чистый оверлап 50 нс на M1/M2, строб 14.0 нс на SiC без звона
STEP 3	Low-V ZVS Verification	Лабораторный БП 3.3–5.0 В, ограничение 0.5 А	Щуп 10:1 на стоках M1/M2: полуволны 1.45 мкс, срабатывание COMP1 и COMP2	Мягкая коммутация ZVS, отсутствие пиковых выбросов выше напряжения питания

Этап ввода	Наименование	Состояние силовой части	Проверяемые точки и измерительные приборы	Критерий перехода к следующему шагу
				более чем в 3.14 раза
STEP 4	HV Ramping	Аккумулятор 2S LiPo, шаг подъема: 100V → 200V → 400V → 780V	Высоковольтный щуп Testec TT-HV 250 (100:1) на C_BUF, контроль синхронного останова	Выключение ШИМ в нуле, амплитуда выбросов на стоках M1/M2 < 35 В, холодные TVS
STEP 5	Single Plasma Strike	Шина 780 В, подача аргона через клапан	Щуп TT-HV 250 на вторичной обмотке T2, регистрация единичного импульса	Пробой зазора, $V_{\text{peak}} \geq$ 7.0 кВ, фронт $t_r \leq 12$ нс, демпфирование звона T2
STEP 6	Full Auto 10 kHz	Полнофункциональный боевой режим	Визуальный контроль плазменной струи в сопле, мониторинг температуры термопарой	Стабильный холодный факел длины 15–20 мм, отсутствие сбоев в работе дисплея OLED

Телеметрия и интерфейс OLED SSD1306 (128x64 SPI)

Конфигурация аппаратного SPI2 и драйвер кадрового буфера

Для предотвращения искажений изображения под воздействием электромагнитных импульсов наносекундного разрядника все линии интерфейса SPI2 экранируются,

снабжаются последовательными демпфирующими резисторами 33 Ом и работают на высокой тактовой частоте передачи данных 21.25 МГц.

C

```
#define SSD1306_WIDTH   128
#define SSD1306_HEIGHT  64

static uint8_t frame_buffer[1024];

static inline void oled_cs_set(bool high)
{
   if (high) GPIOB->BSRR = GPIO_BSRR_BS12;
   else     GPIOB->BSRR = GPIO_BSRR_BR12;
}

static inline void oled_dc_set(bool is_data)
{
   if (is_data) GPIOB->BSRR = GPIO_BSRR_BS14;
   else       GPIOB->BSRR = GPIO_BSRR_BR14;
}

static void spi2_write(uint8_t byte)
{
   while (!(SPI2->SR & SPI_SR_TXE));
   *(volatile uint8_t *)&SPI2->DR = byte;
   while (SPI2->SR & SPI_SR_BSY);
}

void ssd1306_send_command(uint8_t cmd)
{
```

```
    oled_dc_set(false);
    oled_cs_set(false);
    spi2_write(cmd);
    oled_cs_set(true);
}

void ssd1306_system_init(void)
{
    // Аппаратный сброс контроллера дисплея
    GPIOC->BSRR = GPIO_BSRR_BR6;
    for (volatile int i = 0; i < 20000; i++) { __NOP(); }
    GPIOC->BSRR = GPIO_BSRR_BS6;
    for (volatile int i = 0; i < 20000; i++) { __NOP(); }

    // Конфигурация интерфейса SPI2 в Master Mode (Baudrate = 170 МГц / 8 = 21.25 МГц)
    RCC->APB1ENR1 |= RCC_APB1ENR1_SPI2EN;
    SPI2->CR1 = SPI_CR1_SSM | SPI_CR1_SSI | SPI_CR1_MSTR | (2U << SPI_CR1_BR_Pos);
    SPI2->CR2 = (7U << SPI_CR2_DS_Pos); // 8-битный фрейм данных
    SPI2->CR1 |= SPI_CR1_SPE;

    // Последовательность инициализации регистров SSD1306
    ssd1306_send_command(0xAE); // Display Off
    ssd1306_send_command(0xD5); // Set Display Clock Divide Ratio
    ssd1306_send_command(0x80);
    ssd1306_send_command(0xA8); // Set Multiplex Ratio
    ssd1306_send_command(0x3F); // 1/64 duty
    ssd1306_send_command(0xD3); // Set Display Offset
    ssd1306_send_command(0x00);
    ssd1306_send_command(0x40); // Set Start Line
```

```
  ssd1306_send_command(0x8D); // Enable Charge Pump
  ssd1306_send_command(0x14);
  ssd1306_send_command(0x20); // Horizontal Addressing Mode
  ssd1306_send_command(0x00);
  ssd1306_send_command(0xA1); // Segment Re-map
  ssd1306_send_command(0xC8); // COM Output Scan Direction
  ssd1306_send_command(0xDA); // COM Pins Hardware Configuration
  ssd1306_send_command(0x12);
  ssd1306_send_command(0x81); // Set Contrast
  ssd1306_send_command(0xCF);
  ssd1306_send_command(0xD9); // Set Pre-charge Period
  ssd1306_send_command(0xF1);
  ssd1306_send_command(0xDB); // Set VCOMH Deselect Level
  ssd1306_send_command(0x40);
  ssd1306_send_command(0xA4); // Output Follows RAM
  ssd1306_send_command(0xA6); // Normal Display
  ssd1306_send_command(0xAF); // Display On
}

void ssd1306_update_screen(void)
{
  ssd1306_send_command(0x21); // Set Column Address
  ssd1306_send_command(0x00);
  ssd1306_send_command(127);
  ssd1306_send_command(0x22); // Set Page Address
  ssd1306_send_command(0x00);
  ssd1306_send_command(7);

  oled_dc_set(true);
```

```
    oled_cs_set(false);
    for (uint16_t i = 0; i < 1024; i++)
    {
        while (!(SPI2->SR & SPI_SR_TXE));
        *(volatile uint8_t *)&SPI2->DR = frame_buffer[i];
    }
    while (SPI2->SR & SPI_SR_BSY);
    oled_cs_set(true);
}
```

Графический модуль, знакогенератор и компоновка экранов

C

```
static const uint8_t font5x7_ascii[][5] = {
    [' '] = {0x00, 0x00, 0x00, 0x00, 0x00},
    ['0'] = {0x3E, 0x51, 0x49, 0x45, 0x3E},
    ['1'] = {0x00, 0x42, 0x7F, 0x40, 0x00},
    ['2'] = {0x42, 0x61, 0x51, 0x49, 0x46},
    ['3'] = {0x21, 0x41, 0x45, 0x4B, 0x31},
    ['4'] = {0x18, 0x14, 0x12, 0x7F, 0x10},
    ['5'] = {0x27, 0x45, 0x45, 0x45, 0x39},
    ['6'] = {0x3C, 0x4A, 0x49, 0x49, 0x30},
    ['7'] = {0x01, 0x71, 0x09, 0x05, 0x03},
    ['8'] = {0x36, 0x49, 0x49, 0x49, 0x36},
    ['9'] = {0x06, 0x49, 0x49, 0x29, 0x1E},
    [':'] = {0x00, 0x36, 0x36, 0x00, 0x00},
    ['/'] = {0x20, 0x10, 0x08, 0x04, 0x02},
    ['['] = {0x00, 0x7F, 0x41, 0x41, 0x00},
    [']'] = {0x00, 0x41, 0x41, 0x7F, 0x00},
    ['='] = {0x14, 0x14, 0x14, 0x14, 0x14},
    ['.'] = {0x00, 0x40, 0x60, 0x00, 0x00},
```

```
    ['A'] = {0x7E, 0x11, 0x11, 0x11, 0x7E},
    ['B'] = {0x7F, 0x49, 0x49, 0x49, 0x36},
    ['C'] = {0x3E, 0x41, 0x41, 0x41, 0x22},
    ['D'] = {0x7F, 0x41, 0x41, 0x22, 0x1C},
    ['E'] = {0x7F, 0x49, 0x49, 0x49, 0x41},
    ['F'] = {0x7F, 0x09, 0x09, 0x09, 0x01},
    ['G'] = {0x3E, 0x41, 0x49, 0x49, 0x7A},
    ['H'] = {0x7F, 0x08, 0x08, 0x08, 0x7F},
    ['I'] = {0x00, 0x41, 0x7F, 0x41, 0x00},
    ['K'] = {0x7F, 0x08, 0x14, 0x22, 0x41},
    ['L'] = {0x7F, 0x40, 0x40, 0x40, 0x40},
    ['N'] = {0x7F, 0x04, 0x08, 0x10, 0x7F},
    ['O'] = {0x3E, 0x41, 0x41, 0x41, 0x3E},
    ['P'] = {0x7F, 0x09, 0x09, 0x09, 0x06},
    ['R'] = {0x7F, 0x09, 0x19, 0x29, 0x46},
    ['S'] = {0x46, 0x49, 0x49, 0x49, 0x31},
    ['T'] = {0x01, 0x01, 0x7F, 0x01, 0x01},
    ['U'] = {0x3F, 0x40, 0x40, 0x40, 0x3F},
    ['V'] = {0x1F, 0x20, 0x40, 0x20, 0x1F},
    ['Z'] = {0x61, 0x51, 0x49, 0x45, 0x43}
};

void oled_set_pixel(int16_t x, int16_t y, bool on)
{
    if (x < 0 || x >= SSD1306_WIDTH || y < 0 || y >= SSD1306_HEIGHT) return;
    if (on)
        frame_buffer[x + (y / 8) * SSD1306_WIDTH] |= (1U << (y % 8));
    else
        frame_buffer[x + (y / 8) * SSD1306_WIDTH] &= ~(1U << (y % 8));
```

```
}

void oled_put_char(int16_t x, int16_t y, char ch, uint8_t scale)
{
  if (ch < ' ' || ch > 'Z') ch = ' ';
  for (uint8_t i = 0; i < 5; i++)
  {
    uint8_t col = font5x7_ascii[(uint8_t)ch][i];
    for (uint8_t j = 0; j < 8; j++)
    {
      if (col & (1U << j))
      {
        for (uint8_t sx = 0; sx < scale; sx++)
          for (uint8_t sy = 0; sy < scale; sy++)
            oled_set_pixel(x + i * scale + sx, y + j * scale + sy, true);
      }
    }
  }
}

void oled_print_string(int16_t x, int16_t y, const char *str, uint8_t scale)
{
  while (*str)
  {
    oled_put_char(x, y, *str, scale);
    x += 6 * scale;
    str++;
  }
}
```

```
void ui_render_main_screen(const plasma_machine_t *dev)
{
   for (uint16_t i = 0; i < 1024; i++) frame_buffer[i] = 0x00;

   // 1. Верхний информационный бар: Аккумулятор, статус газа, режим работы
   oled_print_string(0, 0, "[====] 7.9V", 1);
   oled_print_string(72, 0, "ARG", 1);
   oled_print_string(96, 0, (dev->state == FSM_STATE_STEP_DEBUG) ? "STEP" : "RUN", 1);

   for (int x = 0; x < 128; x++) oled_set_pixel(x, 9, true);

   // 2. Центральный измерительный блок
   oled_print_string(2, 14, "HV: 780V / 780V", 1);
   oled_print_string(2, 26, "FREQ: 10.0 kHz", 1);
   oled_print_string(2, 38, "PULSE: 14.0 ns", 1);

   // 3. Нижний блок статуса FSM и графический тайм-бар лимита 15.0 секунд
   const char *state_tag = "IDLE";
   switch (dev->state)
   {
      case FSM_STATE_PREFLOW:     state_tag = "PREFLOW"; break;
      case FSM_STATE_CHARGE:      state_tag = "CHARGE";  break;
      case FSM_STATE_FIRING:      state_tag = "FIRING";  break;
      case FSM_STATE_POSTFLOW:    state_tag = "COOLING"; break;
      case FSM_STATE_THERMAL_LOCK: state_tag = "LOCKOUT"; break;
      case FSM_STATE_FAULT:       state_tag = "FAULT";   break;
      default: break;
   }
```

```
    oled_print_string(2, 52, state_tag, 1);

    // Границы индикатора: координата X от 54 до 124, Y от 52 до 60
    for (int x = 54; x <= 124; x++) { oled_set_pixel(x, 52, true); oled_set_pixel(x, 60, true); }
    for (int y = 52; y <= 60; y++) { oled_set_pixel(54, y, true); oled_set_pixel(124, y, true); }

    // Заполнение тайм-бара пропорционально текущему времени непрерывного удержания
    uint8_t bar_fill = (dev->continuous_firing_ms * 68) / 15000;
    if (bar_fill > 68) bar_fill = 68;
    for (int x = 55; x < 55 + bar_fill; x++)
    {
        for (int y = 53; y < 60; y++)
        {
            oled_set_pixel(x, y, true);
        }
    }

    ssd1306_update_screen();
}

void ui_render_debug_screen(uint8_t step)
{
    for (uint16_t i = 0; i < 1024; i++) frame_buffer[i] = 0x00;

    oled_print_string(20, 0, "ENGINEER MENU", 1);
    for (int x = 0; x < 128; x++) oled_set_pixel(x, 9, true);

    switch (step)
```

```
{
  case 0:
    oled_print_string(2, 16, "STEP 0: COLD CHECK", 1);
    oled_print_string(2, 28, "3.3V 15V 5V_ISO", 1);
    oled_print_string(2, 44, "POWER RAILS: OK", 1);
    break;
  case 1:
    oled_print_string(2, 16, "STEP 1: VALVE TEST", 1);
    oled_print_string(2, 28, "FIRE KEY -> SOL1", 1);
    oled_print_string(2, 44, "CHECK AR FLOW", 1);
    break;
  case 2:
    oled_print_string(2, 16, "STEP 2: DRY GATES", 1);
    oled_print_string(2, 28, "345kHz 50ns OVERLAP", 1);
    oled_print_string(2, 44, "SiC PULSE: 14ns", 1);
    break;
  case 3:
    oled_print_string(2, 16, "STEP 3: LOW-V ZVS", 1);
    oled_print_string(2, 28, "VIN: 3.3-5.0V 0.5A", 1);
    oled_print_string(2, 44, "CHECK ZERO CROSS", 1);
    break;
  case 4:
    oled_print_string(2, 16, "STEP 4: HV RAMPING", 1);
    oled_print_string(2, 28, "100V 200V 400V 780V", 1);
    oled_print_string(2, 44, "SYNC STOP VERIFIED", 1);
    break;
  case 5:
    oled_print_string(2, 16, "STEP 5: SINGLE STRIKE", 1);
    oled_print_string(2, 28, "ARGON PLASMA 7.7kV", 1);
```

```
        oled_print_string(2, 44, "CHECK T2 DAMPING", 1);
        break;
      case 6:
        oled_print_string(2, 16, "STEP 6: FULL AUTO", 1);
        oled_print_string(2, 28, "10.0 kHz BURST", 1);
        oled_print_string(2, 44, "COLD JET ACTIVE", 1);
        break;
    }

  ssd1306_update_screen();
}
```

Архитектурный синтез и верификация детерминизма

Проектирование прошивки генератора холодной атмосферной плазмы на базе STM32G474 объединяет программные алгоритмы и кремниевые аппаратные структуры микроконтроллера в единый контур реального времени. За счет калибровки блока DLL таймера HRTIM1 на частоте 5.44 ГГц достигается воспроизведение субмикросекундных и наносекундных импульсов с дискретностью 183.82 пс. Это исключает разброс длительности строба открытого состояния SiC-транзистора Wolfspeed C3M0075120K (строго 14.0 нс), гарантируя согласованную отдачу энергии накопителя C_{BUF} в автотрансформатор T2 без его насыщения.

Мягкая коммутация первичного пуш-пула ZVS на частоте 345 кГц обеспечивается связкой аналоговых компараторов COMP1 и COMP2 с аппаратными каналами внешних событий EEV1 и EEV2. Фазовое опережение переключения транзисторов CSD18534Q5A с оверлапом 50 нс предотвращает возникновение сквозных емкостных токов. Синхронное снятие импульсов накачки, привязанное к моменту сброса счетчиков Master Timer, исключает возникновение импульсов самоиндукции на индуктивности $L_6 = 22$ мкГн, защищая супрессоры 1.5SMC39A от теплового пробоя.

Аппаратная линия защиты Fault 1 блокирует генерацию импульсов менее чем за 25 нс при превышении порога 900 В, обеспечивая безопасность силовой ступени независимо от состояния процессорного ядра. Разделение контуров заземления на системную и плавающую высоковольтную землю с соединением в монолитном узле Net Tie, вкупе с аппаратной изоляцией измерительного тракта через усилитель AMC1311, гарантирует бесперебойную работу пользовательского интерфейса и управляющей логики в условиях жестких электромагнитных помех наносекундных стримерных разрядов.

1. Анализ осциллограмм QSPICE: подтверждение ZVS и наносекундного пробоя

1. **Осциллограмма 4 — Резонансная накачка (V(dr_1), V(dr_2)):**
    - **Идеальный ZVS:** Форма напряжения на стоках M_1 и M_2 представляет собой чистые полусинусоиды с естественным спадом до нуля перед очередным включением ключа. Амплитуда полуволн в установившемся режиме составляет 30–32 В, что с хорошим запасом укладывается в ОБР 60-вольтовых транзисторов CSD18534Q5A и не достает до напряжения пробоя защитных супрессоров 1.5SMC39A ($V_{BR} = 39$ В).
    - **Время заряда накопителя:** Выход на целевой потенциал 800 В на буферной емкости $C_{BUF} = 2.0$ нФ занимает ровно 130.5 мкс. Это подтверждает корректность выбранного в прошивке защитного таймаута заряда 250 мкс (он имеет двукратный запас надежности на случай просадки батареи).
    - **Фиксация паразитного выброса 41 В в момент отсечки:** В точке $t \approx 130.6$ мкс виден одиночный выброс напряжения до 41 В, жестко срезанный супрессором 1.5SMC39A. Это прямое доказательство того, почему **синхронный останов по окончании полупериода** в коде HRTIM жизненно необходим: если оборвать накачку асинхронно посреди такта, вся энергия дросселя $L_6 = 22$ мкГн пойдет в супрессоры, вызывая их мгновенный разогрев.
2. **Осциллограмма 3 — Разрядный SiC ключ (V(sic_drain)):**
    - Напряжение шины до импульса составляет 800 В.
    - В момент строба напряжение падает практически до 0 В ровно на 14.0 нс.
    - При запирании ключа индуктивный выброс автотрансформатора T_2 срезается RCD-клампером (диод C4D02120E) на уровне 560 В, после чего плавно релаксирует к уровню 450–500 В. Для транзистора C3M0075120K с предельным напряжением 1200 В это дает более чем двукратный запас по напряжению ($V_{ds,max} = 560$ В $\ll 1200$ В).
3. **Осциллограммы 1 и 2 — Выход на сопло (V(n_nozzle) и производная dV/dt):**
    - Размах выходного импульса составляет от $+3.5$ кВ до -3.5 кВ, давая суммарные 7.0–7.1 кВ$_{pp}$.
    - Производная нарастания фронта $D\big(V(\text{n_nozzle})\big)$ достигает колоссальных величин: от -8.5 ТВ/с до $+8.3$ ТВ/с (что эквивалентно ≈ 8.5 кВ/нс!). Это многократно превышает минимальный порог стримерного пробоя (50–80 В/нс), гарантируя формирование холодной плазмы без термической контракции разряда.

2. Важнейшая кремниевая особенность STM32G474 (RM0440): маппинг EEV

При реализации прошивки на кристалле STM32G474CEU6 существует неочевидная аппаратная ловушка в блоке внешних событий HRTIM (таблица *HRTIM external event sources* в RM0440):

- Входная матрица **EEV1** принимает сигналы: Src1 = вывод PC12, Src2 = **COMP2**, Src3 = TIM1_TRGO, Src4 = ADC1_AWD1. **Компаратор COMP1 физически не может быть источником для EEV1 напрямую через регистр EECR1!**
- Компаратор **COMP1** аппаратно жестко соединен с входом **EEV6**.
- Компаратор **COMP2** подключен к **EEV7** (а также к EEV1 как Src2).
- Компаратор **COMP3** (отсечка 780 В) подключен к **EEV8**.

Если в регистре таймера HRTIM Timer A настроить сброс счетчика от события EEV1, ожидая там сигнал с COMP1 (PA1), компаратор стока M_1 не будет сбрасывать таймер.

Решение на уровне CMSIS:

Для аппаратного автоподхвата резонанса ZVS привязку событий следует сконфигурировать следующим образом:

- Сброс Timer A (ключ M_1): по событию **EEV6** (источник — COMP1).
- Сброс Timer B (ключ M_2): по событию **EEV7** или **EEV1** (источник — COMP2).
- Аппаратный сигнал отсечки: по событию **EEV8** (источник — COMP3).

C

```
// Корректная регистровая привязка источников EEV в HRTIM1
// EEV6: COMP1 (сброс Timer A), EEV7: COMP2 (сброс Timer B)
HRTIM1->sCommonRegs.EECR2 |= (0U << HRTIM_EECR2_EE6SRC_Pos) | (0U << HRTIM_EECR2_EE7SRC_Pos);
HRTIM1->sCommonRegs.EEFR2 |= (1U << HRTIM_EEFR2_EE6FLTR_Pos) | (1U << HRTIM_EEFR2_EE7FLTR_Pos);


// Привязка сброса к таймерам A и B
HRTIM1->sTimerxRegs[0].RSTAR = HRTIM_RSTAR_EEV6; // Сброс по спаду стока M1 (COMP1)
HRTIM1->sTimerxRegs[1].RSTAR = HRTIM_RSTAR_EEV7; // Сброс по спаду стока M2 (COMP2)
```

3. Верификация распиновки платы WeAct Studio STM32G474CEU6

Все сигналы из схемы полностью согласованы с 48-выводным корпусом UFQFPN48 платы WeAct Studio:

1. **PA8, PA10, PA11 (HRTIM1):** Все три пина разведены на колодки платы WeAct и поддерживают альтернативную функцию AF13, обеспечивая прямой выход стробов 5.44 ГГц без пересечения со служебными цепями (USB/SWD).
2. **PA1, PA7, PA2, PA3, PA0 (Аналоговые входы):** Подключены напрямую к внутренним скоростным компараторам и дифференциальным входам ADC1 без внутренних конфликтов коммутации.
3. **PB12, PB13, PB14, PB15, PC6 (Дисплей SSD1306):** Шина SPI2 и линии управления свободны от встроенных подтяжек платы; вывод PC6 (RST) на плате WeAct разведен на внешнюю гребенку и не задействован под пользовательский светодиод (на плате WeAct светодиод находится на PC13).
4. **PB4, PB5, PB2, PB3 (Энкодер и кнопки):** Каналы таймера TIM3 AF2 снабжены внутренними аппаратными фильтрами ICxF = 0xF, что исключает ложные срабатывания от емкостных наводок при наносекундных высоковольтных разрядах.

4. Практическое замечание по частоте повторения 10 кГц

Из графика симуляции видно, что полный заряд шины с 0 В до 800 В занимает 130.5 мкс. При работе в режиме непрерывной плазменной струи на частоте 10 кГц период между выстрелами составляет 100 мкс. Это означает, что в боевом режиме:

- Буферная емкость $C_{\text{BUF}} = 2.0$ нФ разряжается импульсом SiC лишь частично (до ~ 450–500 В, как видно на графике 3).
- Для восстановления потенциала до 780 В контуру накачки требуется не 130 мкс, а всего около 35–45 мкс подкачки, что с запасом укладывается в 100-микросекундный такт генерации 10 кГц.

Код из кьюспайс:

```
#include <math.h>

union uData {
   bool b; char c; unsigned char uc; short s; unsigned short us;
   int i; unsigned int ui; float f; double d;
   long long int i64; unsigned long long int ui64;
   char *str; unsigned char *bytes;
};

int __stdcall DllMain(void *module, unsigned int reason, void *reserved) {
   (void)module;
   (void)reason;
   (void)reserved;
   return 1;
}
```

```
enum State {
   STATE_CHARGE = 0,
   STATE_HOLD,
   STATE_BLANK_PRE,
   STATE_FIRE_SIC,
   STATE_BLANK_POST
};

struct sINSTANCE {
   State  state;
   double t_state_start;
   double t_cycle_start;
   double target_hv;
   double t_last_switch;
   int    active_leg;
```

```
    bool   stop_charge_pending;

};


extern "C" __declspec(dllexport) void jet_stream(void **opaque, double t, union uData
*data) {

    sINSTANCE *inst = (sINSTANCE *)*opaque;

    if (!inst) {

        inst = new sINSTANCE();

        inst->state             = STATE_CHARGE;

        inst->t_state_start     = 0.0;

        inst->t_cycle_start     = 0.0;

        inst->t_last_switch     = 0.0;

        inst->active_leg        = 1;

        inst->stop_charge_pending = false;


        // Target 780V on HV bus

        inst->target_hv         = 800.0;
```

```
   *opaque = (void *)inst;

}


// Pin mapping

double  V_HV    = data[0].d; // Storage capacitor C_BUF

double  V_DR2   = data[1].d; // Drain M2 feedback

double  V_DR1   = data[2].d; // Drain M1 feedback

double &b_g1    = data[3].d; // Gate M1 (0.0 / 1.0 V)

double &c_g2    = data[4].d; // Gate M2 (0.0 / 1.0 V)

double &OUT_SIC = data[5].d; // Gate SiC (0.0 / 1.0 V)


// Hardware tank timings matched to actual 1.45 us half-wave (345 kHz)

const double T_HALF_NATURAL = 1.45e-6;       // 1.45 us half-period

const double T_MIN_PULSE    = 0.90e-6;       // Blanking gate noise

const double T_MAX_TIMEOUT  = 1.65e-6;       // Fallback switch threshold

const double T_OVERLAP      = 50.0e-9;       // 50 ns current-fed overlap
```

```
const double V_ZVS_THRESH   = 2.0;          // Zero-crossing comparator threshold


// Repetition sequence

const double T_BLANK_PRE    = 1.0e-6;       // 1 us pre-shot deadtime

const double T_PULSE_SIC    = 14.0e-9;      // 14 ns SiC stroke

const double T_BLANK_POST   = 2.0e-6;       // 2 us plasma relaxation

const double T_CYCLE_REP    = 100.0e-6;     // 10 kHz master cycle (100 us)


switch (inst->state) {


  case STATE_CHARGE: {

    OUT_SIC = 0.0;


    if (V_HV >= inst->target_hv) {

      inst->stop_charge_pending = true;

    }


    double t_leg = t - inst->t_last_switch;
```

```
// ZVS detection strictly after blanking window

bool zvs_event = false;

if (t_leg >= T_MIN_PULSE) {

   if (inst->active_leg == 1 && V_DR2 <= V_ZVS_THRESH) {

      zvs_event = true;

   } else if (inst->active_leg == 2 && V_DR1 <= V_ZVS_THRESH) {

      zvs_event = true;

   }

}


// Fallback timeout

if (t_leg >= T_MAX_TIMEOUT) {

   zvs_event = true;

}
```

```
if (zvs_event) {

   if (inst->stop_charge_pending) {

      b_g1 = 0.0;

      c_g2 = 0.0;

      inst->stop_charge_pending = false;

      inst->state       = STATE_HOLD;

      inst->t_state_start = t;

      break;

   }


   // Switch active leg

   inst->active_leg    = (inst->active_leg == 1) ? 2 : 1;

   inst->t_last_switch = t;

   t_leg           = 0.0;

}
```

```
    // Current-fed gate signals with overlap

    if (inst->active_leg == 1) {

      b_g1 = 1.0;

      c_g2 = (t_leg < T_OVERLAP) ? 1.0 : 0.0;

    } else {

      c_g2 = 1.0;

      b_g1 = (t_leg < T_OVERLAP) ? 1.0 : 0.0;

    }

    break;

}


case STATE_HOLD: {

    b_g1   = 0.0;

    c_g2   = 0.0;

    OUT_SIC = 0.0;
```

```
    if ((t - inst->t_cycle_start) >= (T_CYCLE_REP - T_BLANK_PRE)) {

      inst->state       = STATE_BLANK_PRE;

      inst->t_state_start = t;

    }

    break;

  }

  case STATE_BLANK_PRE: {

    b_g1    = 0.0;

    c_g2    = 0.0;

    OUT_SIC = 0.0;

    if ((t - inst->t_cycle_start) >= T_CYCLE_REP) {
      inst->state       = STATE_FIRE_SIC;
      inst->t_state_start = t;

      while ((t - inst->t_cycle_start) >= T_CYCLE_REP) {
        inst->t_cycle_start += T_CYCLE_REP;
      }
    }
```

```
    break;
  }

  case STATE_FIRE_SIC: {

    b_g1    = 0.0;

    c_g2    = 0.0;

    OUT_SIC = 1.0;


    if ((t - inst->t_state_start) >= T_PULSE_SIC) {

      OUT_SIC          = 0.0;

      inst->state      = STATE_BLANK_POST;

      inst->t_state_start = t;

    }

    break;

  }

  case STATE_BLANK_POST: {
```

```
        b_g1   = 0.0;

        c_g2   = 0.0;

        OUT_SIC = 0.0;


        if ((t - inst->t_state_start) >= T_BLANK_POST) {

          inst->state            = STATE_CHARGE;

          inst->t_state_start    = t;

          inst->t_last_switch    = t;

          inst->active_leg       = 1;

          inst->stop_charge_pending = false;

        }

        break;

      }

    }

}
```